

Sistemas Multiagentes (PPGCA-CT e PPGCC-PG)

Intelligent Agents

Prof^a Celia Ghedini Ralha

Universidade Tecnológica Federal do Paraná (UTFPR)
Curitiba/PR, Brasil

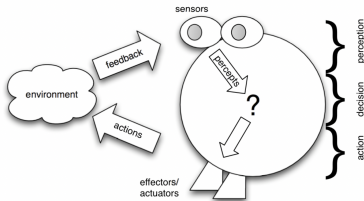
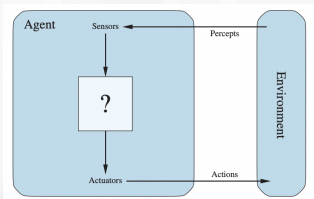
25 de setembro de 2026

Agenda¹

- Agents
- Rationality
- Task environment (PEAS)
- Environment types
- Agent types

¹ S. Russel & P. Norvig. AI: A Modern Approach, 4th US ed., 2020 - Chap. 2

Agents



An agent is a computer system capable of autonomous action in some environment, in order to achieve its delegated goals.²

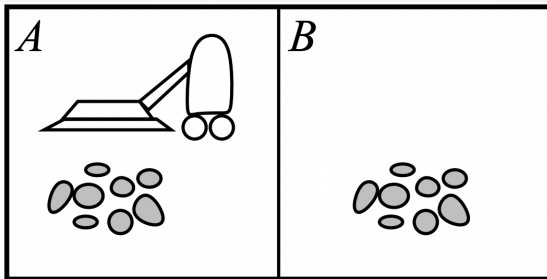
Agents include humans, robots, softbots, thermostats, etc.

The agent function maps from percept histories to actions:

$$f : \mathcal{P}^* \rightarrow \mathcal{A}$$

The agent program runs on the physical architecture to produce f

Vacuum-cleaner world



Percepts: location and contents, e.g., $[A, \textit{Dirty}]$

Actions: *Left*, *Right*, *Suck*, *NoOp*

A vacuum-cleaner agent

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

A vacuum-cleaner agent

What is the **right** function to implement

$$f : \mathcal{P}^* \rightarrow \mathcal{A}$$

Can it be implemented in a small agent program?

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
```

```
  if status = Dirty then return Suck  
  else if location = A then return Right  
  else if location = B then return Left
```

Fixed performance measure evaluates the environment sequence

- one point per square cleaned up in time T ?
- one point per clean square per time step, minus one per move?
- penalize for $> k$ dirty squares?

Rationality

A **rational agent** chooses whichever action maximizes the **expected** value of the performance measure **given the percept sequence to date**

Rational $\neq$ omniscient

- percepts may not supply all relevant information

Rational $\neq$ clairvoyant

- action outcomes may not be as expected

Hence, rational $\neq$ successful

Rational $\rightarrow$ exploration, learning, autonomy

To design a **rational agent**, we must specify the **task environment**

- Performance measure?
- Environment?
- Actuators?
- Sensors?

PEAS: Internet shopping agent



Consider the task of designing an internet shopping agent

- Performance measure - price, quality, appropriateness, efficiency
- Environment - current and future WWW sites, vendors, shippers
- Actuators - display to user, follow URL, fill in form
- Sensors - HTML pages (text, graphics, scripts)

³ Illustration cyborg, Crédito: AI-generated image created by migstc1, available on Freepik, Direitos autorais: migstc1

PEAS: Automated taxi driver



4

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Environment types

There are six different environment types.

The environment type largely determines the agent design

- 1 Fully observable vs. partially observable
- 2 Deterministic vs. nondeterministic
- 3 Episodic vs. sequential
- 4 Static vs. dynamic
- 5 Discrete vs. continuous
- 6 Single vs. multi-agent

Fully vs. Partially observable

- A fully observable environment is one in which the agent can obtain complete, accurate, up-to-date information about the environment's state.
- Most moderately complex environments (including the everyday physical world and the Internet) are partially observable.
- The more accessible an environment is, the simpler it is to build agents to operate in it.

Deterministic vs. nondeterministic

- In a deterministic environment any action has a single guaranteed effect — there is no uncertainty about the state that will result from performing an action.
- The physical world can to all intents and purposes be regarded as nondeterministic.
- Stochastic if it explicitly deals with probabilities (e.g., there's a 25% chance of rain tomorrow). Nondeterministic if the possibilities are listed without being quantified (e.g., there's a chance of rain tomorrow)
- Nondeterministic environments present greater problems for the agent designer.

Episodic vs. Sequential



- In an episodic task environment, agent's experience is divided into atomic episodes. In each episode the agent receives a percept and then performs sequential a single action. Crucially, the next episode does not depend on the actions taken in previous episodes.
- In sequential environments, short-term actions can have long-term consequences (e.g., chess and taxi driving).
- Episodic environments are simpler from the agent developer's perspective because the agent can decide what action to perform based only on the current episode — it need not reason about the interactions between this and future episodes.

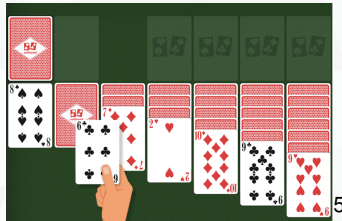
Static vs. Dynamic

- If the environment can change while an agent is deliberating, then we say the environment is dynamic for that agent; otherwise, it is static.
- A static environment is one that can be assumed to remain unchanged except by the performance of actions by the agent.
- A dynamic environment is one that has other processes operating on it, and which hence changes in ways beyond the agent's control.
- If the environment itself does not change with the passage of time but the agent's performance score does, then we say the environment is semidynamic.
- The physical world is a highly dynamic environment.

Discrete vs. Continuous

- An environment is discrete if there are a fixed, finite number of actions and percepts in it.
- The discrete/continuous distinction applies to the state of the continuous environment, to the way time is handled, and to the percepts and actions of the agent.
- Russell & Norvig give a chess game as an example of a discrete environment, and taxi driving as an example of a continuous one.

Environment types: Solitaire Game



The solitaire game environment

- Fully observable
- Deterministic⁶
- Sequential
- Static
- Discrete
- Single agent

5

By Neal Taparia - 06/27/2022 (<https://solitaired.com/guides/how-to-play-solitaire>)

6

A game is deterministic if the resolution of player actions leads to completely predictable outcomes. Truly deterministic games have the same setup every game, present scripted results for each possible decision (e.g., chess).

Environment types: Backgammon Game⁸



The solitaire game environment

- Fully observable
- Stochastic
- Sequential
- Static
- Discrete
- Multi-agent

7

Classic Backgammon: Dice Board Game of Blackout Lab available Google Play (<https://play.google.com/store/apps/details?id=com.blackout.backgammon&hl=en>)

8

One of the most popular dice board game. The objective is to be the first player to move all the checkers into their home board and bear them off.

Environment types: Internet Shopping Agent



The internet shopping environment

- Partially observable
- Partly deterministic
- Sequential
- Semi static
- Discrete
- Single agent or Multi-agent (auctions)

Environment types

The autonomous taxi and real world applications are

- Partially observable
- Stochastic
- Sequential
- Dynamic
- Continuous
- Multi-agent

	Solitaire	Backgammon	Internet shopping	Taxi
<u>Observable??</u>	Yes	Yes	No	No
<u>Deterministic??</u>	Yes	No	Partly	No
<u>Episodic??</u>	No	No	No	No
<u>Static??</u>	Yes	Semi	Semi	No
<u>Discrete??</u>	Yes	Yes	Yes	No
<u>Single-agent??</u>	Yes	No	Yes (except auctions)	No

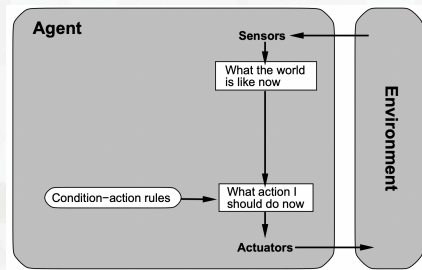
Agent types

There are several basic agent types to increasing generality:

- 1 simple reflex agents
- 2 reflex agents with state
- 3 goal-based agents
- 4 utility-based agents
- 5 learning agents

All these can be turned into learning agents

Simple reflex agent



function SIMPLE-REFLEX-AGENT(*percept*) **return** an action
persistent: *rules*, a set of condition-action rules

state $\leftarrow$ INTERPRET-INPUT (*percept*)

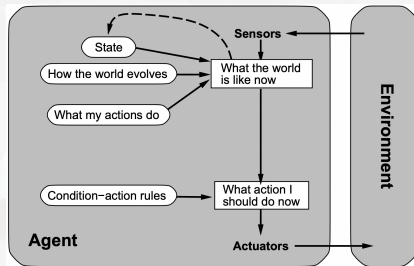
rule $\leftarrow$ RULE-MATCH(*state*,*rules*)

action $\leftarrow$ *rule*.ACTION

return *action*

end function

Reflex agent with state



function REFLEX-AGENT-WITH-STATE(percept) **return** an action
persistent: *state*, the agent's current conception of the world state
model, a description of how next state depends on current state & action
rules, a set of condition-action rules
action, the most recent action, initially none

state $\leftarrow$ UPDATE-STATE (*state*,*action*,*percept*,*model*)

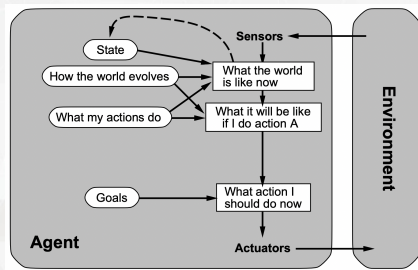
rule $\leftarrow$ RULE-MATCH(*state*,*rules*)

action $\leftarrow$ *rule*.ACTION

return *action*

end function

Goal-based agents



function GOAL-BASED-AGENT(percept) **return** an action

persistent: *state*, the agent's current conception of the world state

model, a description of how next state depends on current state & action

goal, information that describes situations that are desirable, initially a goal

possible-goals, a set of goals the agent needs to accomplish

action, the most recent action, initially none

state $\leftarrow$ UPDATE-STATE (*state*,*action*,*percept*,*model*)

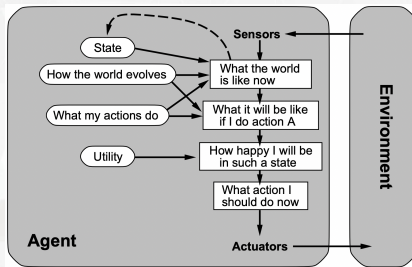
goal $\leftarrow$ UPDATE-GOAL (*state*,*possible-goals*)

action $\leftarrow$ BEST-ACTION (*state*,*goal*)

return *action*

end function

Utility-based agents



function UTILITY-BASED-AGENT(*percept*) **return** an action
persistent: *state*, the agent's current conception of the world state
possible-states, states that may maximize happiness
rules, a set of condition-action rules
utility, function to measure the agent's performance
action, the most recent action, initially none

state $\leftarrow$ UPDATE-STATE (*state*,*action*,*percept*,*possible-states*)

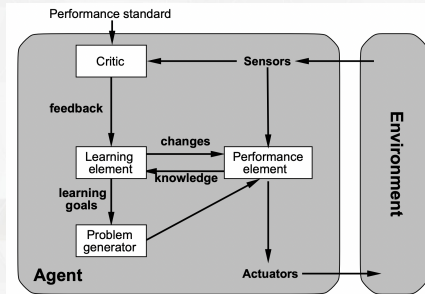
rule $\leftarrow$ RULE-MATCH (*state*,*rules*,*utility*)

action $\leftarrow$ *rule*.ACTION

return *action*

end function

Learning agents



function LEARNING-AGENT(percept,performance-standard) **return** an action

persistent: *critic*, evaluate how the agent is doing

learning-element, responsible for making improvements

performance-element, responsible for selecting the best actions

problem-generator, suggest actions to new & informative experiences

action, the most recent action, initially none

critic $\leftarrow \leftarrow$ INTERPRET-INPUT (*percept*,*performance-standard*)

learning-element $\leftarrow \leftarrow$ FEEDBACK (*critic*)

performance-element $\leftarrow \leftarrow$ CHANGES (*learning-element*)

learning-element $\leftarrow \leftarrow$ KNOWLEDGE (*performance-element*)

problem-generator $\leftarrow \leftarrow$ LEARNING-GOALS (*learning-element*)

performance-element $\leftarrow \leftarrow$ BEST-GOAL (*problem-generator*)

action $\leftarrow$ *performance-element.ACTION*

return *action*

end function

Summary

- **Agents** interact with **environments** through **actuators** and **sensors**
- The **agent function** describes what the agent does in all circumstances
- The **performance measure** evaluates the environment sequence
- A **perfectly rational** agent maximizes expected performance
- **Agent programs** implement agent functions
- **PEAS** descriptions define task environments
- Environments are categorized along several dimensions:
 - ▶ fully vs. partially observable
 - ▶ deterministic vs. stochastic
 - ▶ episodic vs. sequential
 - ▶ static vs. dynamic
 - ▶ discrete vs. continuous
 - ▶ single-agent vs. multi-agent
- Several agent types exist:
 - ▶ simple reflex
 - ▶ reflex with state
 - ▶ goal-based
 - ▶ utility-based
 - ▶ learning

Thank you for coming!



Prof^a Celia Ghedini Ralha
celiaghedini@utfpr.edu.br